Team ID : SWTID-2026-7205
Team Size : 5
Team Leader : DEVADHARSHINI.S
Team member : SARANYA.V
Team member : VARSHA.K
Team member : REENA.S
Team member : BHUVANESWARI.V

Leaf Health Check

Project Description

Monitoring plant health is a crucial activity in agriculture, horticulture, and home gardening. Traditional methods of detecting plant diseases require manual inspection, expert knowledge, and significant time investment. In many cases, early symptoms such as discoloration, spots, or texture changes on leaves are overlooked, leading to severe plant damage and reduced crop productivity. **Leaf Health Check** addresses this challenge by providing an AI-powered plant leaf analysis platform that delivers fast, accurate, and intelligent diagnosis.

Leaf Health Check is a smart application developed using the Streamlit framework that allows users to upload an image of a plant leaf and receive an instant health assessment. The system applies advanced image processing and deep learning techniques to examine visual symptoms such as yellowing, browning, and dark patches. These symptoms are analyzed through a discoloration detection logic that calculates the affected area and identifies possible signs of disease or nutrient deficiency.

The platform also includes a plant species identification module that compares the uploaded leaf image with a structured plant database. Once the plant type is recognized, the system performs disease prediction and evaluates the severity of the condition. Based on the percentage of damage and detected patterns, the leaf is classified into categories such as **Healthy, Mild, Moderate, Severe, or Dying**.

To make the system practical and user-friendly, Leaf Health Check generates a clear diagnosis along with **three personalized rescue tips**. These recommendations guide users in taking corrective actions such as adjusting watering schedules, applying fertilizers, improving sunlight exposure, or using suitable treatments.

By integrating artificial intelligence with an interactive web interface, the project transforms complex plant health monitoring into a simple, accessible, and real-time solution. The system is designed to assist farmers, students, researchers, and gardening enthusiasts in maintaining healthy plants without requiring expert-level knowledge. Leaf Health Check

contributes to smart agriculture and sustainable plant care by enabling early detection and timely intervention.

Use Case Scenarios

Scenario 1: Leaf Image Upload and Health Analysis

Users upload a plant leaf image through the Streamlit interface. The system preprocesses the image by enhancing brightness, resizing it, and removing noise for accurate analysis. The AI model then detects discoloration patterns and predicts whether the leaf is healthy or affected by a disease. The result is displayed along with the severity level and confidence score.

Scenario 2: Plant Species Identification and Disease Detection

The system matches the uploaded leaf image with a plant species database. After identifying the plant type, it analyzes common diseases associated with that species and predicts the most probable condition. This ensures that the diagnosis is plant-specific and more accurate.

Scenario 3: Severity Grading and Rescue Recommendations

Based on the percentage of damaged area and the type of discoloration detected, the system assigns a severity level ranging from Healthy to Dying. It then generates three intelligent rescue tips that help users take immediate action to improve plant health and prevent further damage.

Architecture Overview

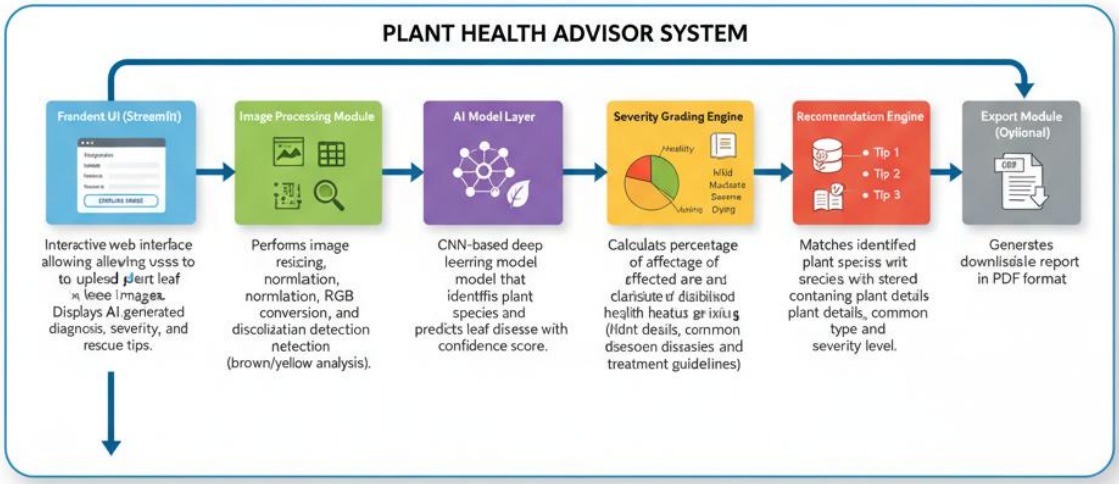
Leaf Health Check is an intelligent AI-powered plant disease detection system designed to analyze plant leaf images and provide accurate health diagnosis with actionable rescue recommendations. The application is built using the **Streamlit framework** and integrates a deep learning-based computer vision model to detect leaf discoloration, identify plant species, and determine disease severity.

The system processes uploaded leaf images, performs color analysis and species recognition, and delivers a structured diagnosis report including severity grading and personalized rescue tips.

Core Technologies

- **Streamlit**: Web application framework for building interactive AI-based user interfaces
- **Python 3.8+**: Backend processing and AI model integration
- **TensorFlow / PyTorch**: Deep learning framework for plant disease classification
- **OpenCV**: Image preprocessing and discoloration detection
- **NumPy & Pandas**: Data processing and analysis
- **SQLite / PostgreSQL**: Plant species and disease database management
- **Matplotlib / PIL**: Image visualization and processing

Component-Wise Architectures
for Plant Leaf Disease Analysis



Component-Wise Architectures

Component	Description
Frontend UI (Streamlit)	Interactive web interface allowing users to upload plant leaf images. Displays AI-generated diagnosis, severity grading, and rescue tips.
Image Processing Module	Performs image resizing, normalization, RGB conversion, and discoloration detection (brown/yellow analysis).
AI Model Layer	CNN-based deep learning model that identifies plant species and predicts leaf disease with confidence score.

Component	Description
Severity Grading Engine	Calculates percentage of affected area and classifies health status (Healthy, Mild, Moderate, Severe, Dying).
Species Database Lookup	Matches identified plant species with stored database containing plant details, common diseases, and treatment guidelines.
Recommendation Engine	Generates three personalized rescue tips based on disease type and severity level.
Export Module (Optional)	Generates downloadable health report in PDF format.

System Workflow

1. User uploads leaf image through Streamlit interface
2. Image preprocessing (resize, normalize, noise reduction)
3. Discoloration detection (yellow/brown pixel percentage analysis)
4. AI model predicts:
    - Plant species
    - Disease class
    - Confidence score
5. Severity grading is calculated
6. Rescue tips are retrieved from database
7. Final diagnosis report is displayed

Severity Grading Logic

- 0–10% Discoloration → **Healthy**
- 11–30% → **Mild**
- 31–60% → **Moderate**
- 61–80% → **Severe**
- 81–100% → **Dying**

Severity is calculated based on:

- Percentage of yellow/brown regions
- Disease probability score
- Leaf texture damage

Output Format

Diagnosis Report

Plant Name: <Identified Plant>
Disease: <Predicted Disease>
Confidence Score: <Model Accuracy %>
Severity Level: <Health Grade>

Rescue Tips:

1. <Tip 1>
2. <Tip 2>

3. <Tip 3>

Deployment Architecture (Streamlit Based)
User → Streamlit UI → Image Processing → AI Model → Severity Engine → Recommendation Engine → Output Display
The application can be deployed on:

- Streamlit Cloud
- AWS / Azure
- Local Server

Pre-requisites

1. Python 3.8 or above
2. Streamlit Framework
3. TensorFlow / PyTorch
4. OpenCV
5. Required Libraries (requirements.txt)

Example installation:

```
pip install -r requirements.txt
```

Run the application:

```
streamlit run app.py
```

Project Workflow

1. AI Model Setup & Initialization

Activity 1.1: Setup Deep Learning Environment

- Install Python and required AI libraries
- Configure TensorFlow / PyTorch environment
- Download plant leaf disease dataset
- Verify GPU/CPU compatibility for model training

Activity 1.2: Configure Environment and Dependencies

- Create virtual environment
- Install required packages using requirements.txt
- Configure model loading paths
- Setup image preprocessing configurations

Activity 1.3: Prepare Detection Logic

- Design discoloration detection algorithm
- Define yellow and brown color thresholds
- Prepare plant species classification labels
- Test preprocessing with sample leaf images

2. Core Functionality Development

Activity 2.1: Design Streamlit Application Structure

- Create main Streamlit interface (app.py)
- Build modular folder structure
- Implement navigation and layout components
- Configure session state handling
-

Activity 2.2: Implement Image Upload Module

- Upload JPG/PNG leaf images
- Validate image size and format
- Display preview of uploaded image
- Add loading indicator during analysis

Activity 2.3: Develop AI Prediction Pipeline

- Perform image preprocessing
- Run species identification model
- Detect disease class
- Calculate prediction confidence score

3. Backend Processing Implementation

Activity 3.1: Image Processing Module

- Resize and normalize images
- Convert images to RGB format
- Remove noise using filtering
- Extract discoloration regions using OpenCV

Activity 3.2: Severity Grading Engine

- Calculate percentage of affected pixels
- Apply grading logic:
    - Healthy

    - Mild
    - Moderate
    - Severe
    - Dying
- Combine discoloration + AI prediction results

Activity 3.3: Species Database Integration

- Create plant species database
- Map detected species with diseases
- Store treatment and care information
- Retrieve matching plant details

Activity 3.4: Recommendation Engine

- Generate rescue tips based on disease
- Adjust tips according to severity level
- Ensure minimum three actionable suggestions

4. UI Development (Streamlit)

Activity 4.1: Build Interactive Interface

- Application title and description
- Image upload widget
- Analyze button
- Progress spinner during processing

Activity 4.2: Implement Result Display

- Show detected plant species
- Display disease diagnosis
- Show severity badge
- Display confidence percentage
- Present rescue tips clearly

Activity 4.3: Add Export Functionality (Optional)

- Generate downloadable diagnosis report
- PDF export option
- File naming and download handling

5. Testing & Optimization

Activity 5.1: Functional Testing

- Test with multiple plant species images
- Validate discoloration detection
- Verify severity grading accuracy

- Check UI responsiveness

Activity 5.2: Model Evaluation

- Measure prediction accuracy
- Validate classification confidence
- Reduce false predictions
- Improve preprocessing quality

Activity 5.3: Performance Optimization

- Optimize image processing speed
- Reduce model loading time
- Improve Streamlit responsiveness
- Handle invalid or low-quality inputs

Activity 5.4: Deployment Readiness

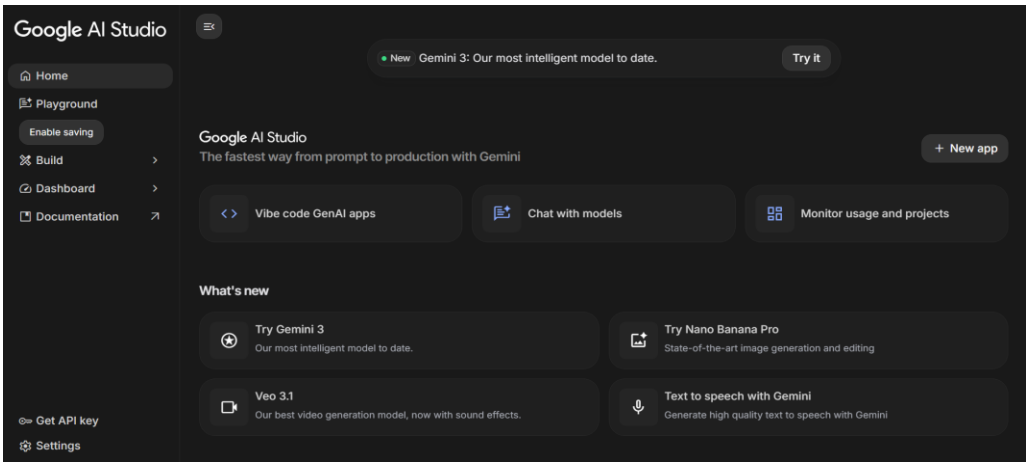
- Prepare requirements.txt
- Configure Streamlit Cloud deployment
- Test application in production environment
- Ensure cross-platform compatibility

MILESTONE 1: Gemini AI Initialization

MILESTONE 1: AI Model Initialization

In this foundational stage, the objective is to establish and configure the core AI infrastructure required for the **Leaf Health Check** system. The platform utilizes a deep learning–based computer vision model to analyze plant leaf images, detect discoloration patterns, identify plant species, and generate health diagnosis with rescue recommendations.

Activity 1.1: Setup Development Environment and AI Model



1. Install Required Software

- Install Python (version 3.8 or above)
- Install required IDE (VS Code / PyCharm)
- Setup virtual environment for project isolation

2. Install Required Libraries

```
pip install streamlit opencv-python tensorflow numpy pandas pillow python-dotenv
```

3. Prepare AI Model

- Download plant leaf disease dataset
- Configure trained CNN model (TensorFlow/PyTorch)
- Store trained model file inside /model directory
- Verify model loading functionality

Activity 1.2: Configure Environment and Test Setup

1. Create. env File

```
MODEL_PATH=model/leaf_model.h5
SECRET_KEY=leaf_health_secret
```

2. Load Environment Variables

Example configuration code:

```
import os
from dotenv import load_dotenv

load_dotenv()

MODEL_PATH = os.getenv("MODEL_PATH")
```

3. Test Model Loading

```
from tensorflow.keras.models import load_model

model = load_model(MODEL_PATH)
print("Model loaded successfully")
```

Expected Result:

- Model loads without errors
- Ready for prediction testing

Activity 1.3: Validate Image Processing and Prediction

Test Image Preprocessing

```
import cv2

image = cv2.imread("sample_leaf.jpg")
image = cv2.resize(image, (224,224))
print("Image preprocessing successful")
```

Test Disease Prediction

```
prediction = model.predict(image.reshape(1,224,224,3))
print("Prediction Generated")
```

Expected Output:

- Model predicts plant disease class

- Confidence score generated successfully

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core application using the **Streamlit framework** integrated with the **Google Gemini API**. The system accepts plant leaf images, processes visual information, analyzes discoloration patterns, and generates an AI-based diagnosis with severity grading and rescue recommendations.

File Explorer Structure

```
Leaf-Health-Check/
|
├── app.py                  # Main Streamlit application
├── requirements.txt        # Project dependencies
├── .env                    # Environment variables
|
├── model/
|   └── preprocess.py       # Image preprocessing logic
|
├── utils/
|   ├── gemini_api.py       # Gemini API integration
|   ├── severity.py         # Severity grading logic
|   ├── recommendations.py  # Rescue tips generator
|
├── assets/
|   └── style.css           # UI styling
|
└── outputs/                # Generated reports
```

Activity 2.1: Streamlit Initialization and Import Libraries

Import Required Libraries

```
import streamlit as st
import google.generativeai as genai
import os
import cv2
import numpy as np
from dotenv import load_dotenv
from PIL import Image
```

Initialize Environment and Gemini API

```
load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("models/gemini-2.5-flash")
```

Activity 2.2: Main Functionality – Leaf Analysis Routing

Image Upload Interface

```
st.title("🌿 Leaf Health Check")

uploaded_file = st.file_uploader(
    "Upload a plant leaf image",
    type=["jpg","png","jpeg"]
)
```

Image Processing

```
if uploaded_file:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Leaf", use_column_width=True)
```

Gemini AI Diagnosis Request

```
prompt = f"""
Analyze the uploaded plant leaf image.

Tasks:
1. Identify plant species
2. Detect discoloration (yellow/brown areas)
3. Predict possible disease
4. Assign severity level (Healthy, Mild, Moderate, Severe, Dying)
5. Provide 3 rescue tips
"""

response = model.generate_content(prompt)
result = response.text
```

Display Result

```
st.subheader("Diagnosis Report")
st.write(result)
```

Activity 2.3: Severity Grading Logic

```python
def calculate_severity(discolor_percent):
    if discolor_percent <= 10:
        return "Healthy"
    elif discolor_percent <= 30:
        return "Mild"
    elif discolor_percent <= 60:
        return "Moderate"
    elif discolor_percent <= 80:
        return "Severe"
    else:
        return "Dying"
```

Activity 2.4: Export Functionality

Generate Text Report

```python
def generate_report(content):
    filename = "outputs/leaf_report.txt"
    with open(filename,"w") as f:
        f.write(content)
    return filename
```

python

```python
if st.button("Download Report"):
    file = generate_report(result)
    st.download_button("Download", open(file))
```

Activity 2.5: Utility Functions

Clean AI Response

```python
def clean_response(text):
    return text.strip()
```

Unique File Generator

```python
from datetime import datetime

def generate_filename():
    return datetime.now().strftime("%Y%m%d_%H%M%S")
```

MILESTONE 3: Backend – Streamlit Integration (app.py)

This milestone focuses on implementing the complete backend logic inside the **Streamlit application**, integrating the **Google Gemini AI Studio API**, handling image processing, performing diagnosis generation, and enabling export functionality. The system ensures proper validation, structured responses, and smooth interaction between the UI and AI services.

Activity 3.1: Complete Function Definitions

All backend processing is implemented with:

- Input image validation
- Error handling mechanisms
- Gemini API response handling
- Logging and debugging support
- Structured diagnosis output formatting

Image Validation Example

```
def validate_image(file):
    if file is None:
        return False, "No image uploaded"
    return True, "Valid image"
```

Activity 3.2: Modular Architecture

Main Application Structure

```python
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("models/gemini-2.5-flash")

st.title("🌿 Leaf Health Check System")

print("="*60)
print("Leaf Health Check - Starting Application")
print("✓ Gemini API Key configured")
print("✓ Using model: gemini-2.5-flash")
print("="*60)
```

The Streamlit application directly manages:

- User interaction
- AI communication
- Diagnosis generation
- Result visualization

Activity 3.3: Gemini AI Diagnosis Integration

Generate Diagnosis from Image

```python
def generate_diagnosis():
    prompt = """
    Analyze the uploaded plant leaf image and provide:
    1. Plant species
    2. Possible disease
    3. Discoloration analysis
    4. Severity level (Healthy, Mild, Moderate, Severe, Dying)
    5. Three rescue tips
    """

    response = model.generate_content(prompt)
    return response.text
```

Display AI Response

```python
if st.button("Analyze Leaf"):
    result = generate_diagnosis()
    st.subheader("Diagnosis Result")
    st.write(result)
```

Activity 3.4: Export Implementation

Complete export functionality includes:

- Diagnosis report generation
- Downloadable text/PDF report
- File management and naming

Report Generation

```python
def export_report(content):
    filename = "leaf_health_report.txt"
    with open(filename, "w") as f:
        f.write(content)
    return filename
```

python

```python
st.download_button(
    label="Download Report",
    data=result,
    file_name="leaf_health_report.txt"
)
```

Activity 3.5: Error Handling and Response Formatting

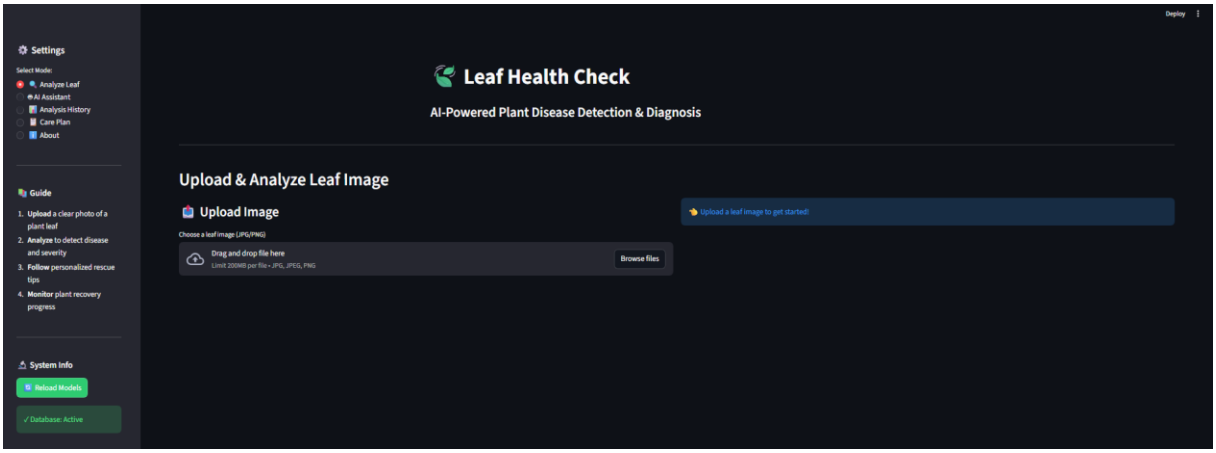
```python
def clean_response(text):
    if text:
        return text.strip()
    return "No response generated"
```

MILESTONE 4: UI Development

This milestone focuses on designing and implementing a **user-friendly and responsive interface** for the **Leaf Health Check** application using the **Streamlit framework**. The UI ensures smooth

interaction between the user and the AI system, enabling easy image upload, analysis execution, and result visualization.

Activity 4.1: Home Page Development



The home page acts as the primary interface for user interaction and provides clear guidance on how to use the application.

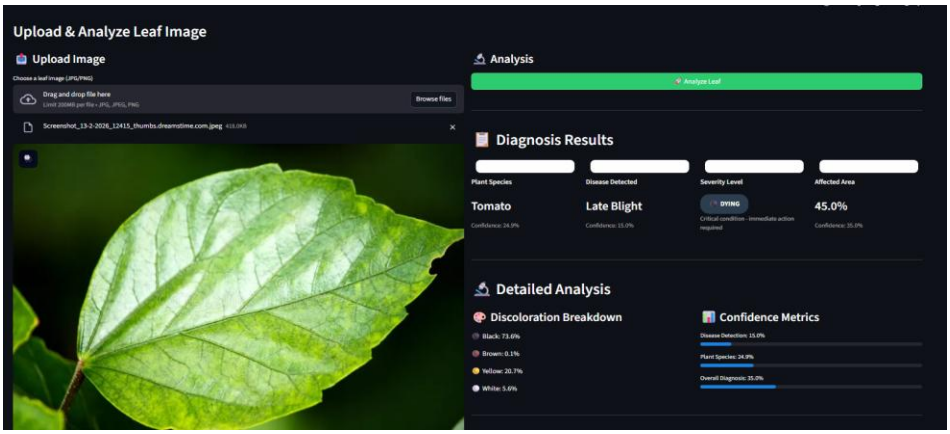
Home Page Features:

- Application title and description
- Leaf image upload section
- Supported image format information
- Analyze button for triggering diagnosis
- Clean and responsive layout

UI Design Considerations:

- Simple and minimal design
- Clear instructions for users
- Responsive layout across screen sizes
- Intuitive navigation flow

Activity 4.2: Leaf Analysis Page



This section handles user input and displays AI-generated results after analysis.

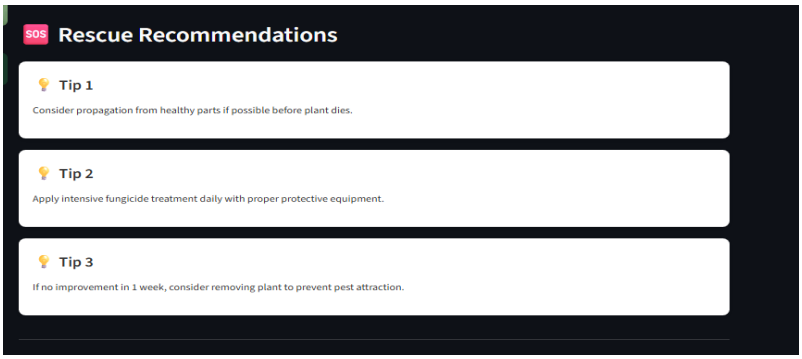
Features Implemented:

- Image preview after upload
- Loading spinner during analysis
- Gemini AI diagnosis display
- Plant species identification
- Disease detection result
- Severity grading badge

Severity Levels Displayed:

- Healthy
- Mild
- Moderate
- Severe
- Dying

Activity 4.3: Rescue Recommendation Display

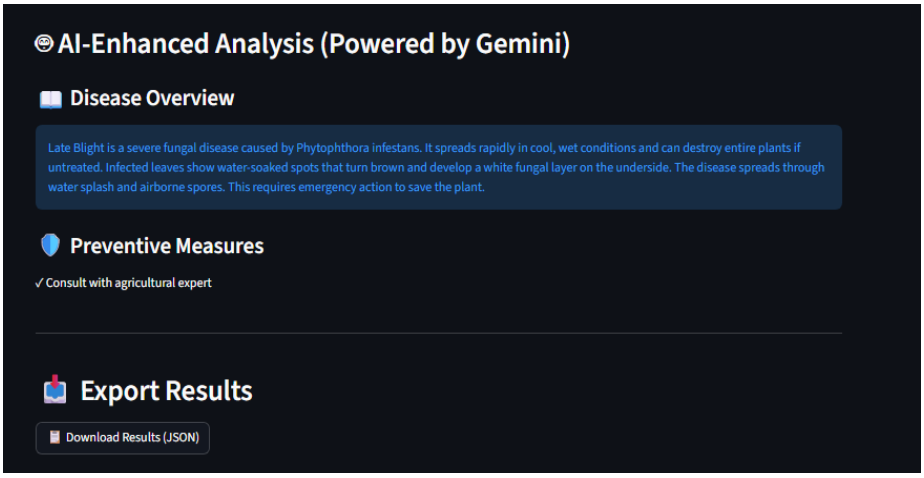


After diagnosis, the system presents **actionable rescue recommendations** to help users treat the affected plant.

Rescue Tips Section Includes:

- Three personalized rescue tips
- Tips based on disease type and severity
- Easy-to-read bullet format
- Practical and actionable suggestions

Activity 4.4: Result Export Option



To enhance usability, the UI provides an option to download the diagnosis report.

Export Features:

- Download diagnosis as a text/PDF report
- Automatically generated report name
- Includes plant name, disease, severity, and tips

Activity 4.5: Error Handling and User Feedback

Proper feedback mechanisms are integrated to improve user experience.

Handled Scenarios:

- Invalid image format
- No image uploaded
- AI response failure
- Low-quality image warning

Feedback Methods:

- Warning messages
- Error alerts
- Success notifications

MILESTONE 5: Testing & Optimization

This milestone focuses on **comprehensive testing, quality evaluation, and performance optimization** of the **Leaf Health Check** application to ensure accuracy, reliability, and smooth user experience. Multiple test scenarios were executed to validate image analysis, AI diagnosis, severity grading, and rescue recommendations.

Activity 5.1: Comprehensive Testing

Test Scenarios

1. Leaf Image Analysis
   - Input Type: Plant leaf images
   - Formats Tested: JPG, PNG, JPEG
   - Conditions: Healthy, diseased, partially damaged leaves
   - Expected Output: Accurate diagnosis with severity grading and rescue tips

Test Cases Executed:

- Healthy green leaf image
- Yellowing leaf image
- Brown spot infected leaf
- Severely damaged leaf image
- Low-quality and blurred images

2. Discoloration Detection
   - Colors Tested: Yellow, brown, black
   - Severity Levels Verified: Healthy, Mild, Moderate, Severe, Dying
   - Expected Result: Correct severity classification based on affected area

3. Gemini AI Diagnosis Testing
   - API Used: Gemini AI Studio API Key
   - Model Tested: Gemini Vision Model
   - Expected Output:
     - Plant species identification
     - Disease prediction
     - Confidence-based severity grading
     - Three actionable rescue tips

4. Error Handling Scenarios
   - No image uploaded

- Unsupported image format
- Corrupted image file
- API response delay or failure

Expected Behavior:

- User-friendly error messages
- Graceful failure handling
- No application crash

Activity 5.2: Quality Evaluation

The output quality of the system was evaluated using the following metrics:

- Diagnosis Accuracy
- Severity Classification Correctness
- Relevance of Rescue Tips
- Response Consistency
- User Interface Clarity
- Report Export Quality

Each test output was manually reviewed to ensure **logical correctness** and **practical usefulness**.

Activity 5.3: Performance Optimization

Improvements Made

- Faster image preprocessing using optimized resizing
- Reduced Gemini API response latency
- Improved input validation logic
- Optimized Streamlit layout rendering
- Added loading indicators for better user feedback
- Reduced unnecessary re-computation using session state

Activity 5.4: Final Validation

After optimization, the system was re-tested to confirm:

- Stable application performance
- Accurate AI predictions
- Correct severity grading
- Smooth UI interaction
- Successful report downloads

Conclusion

The **Leaf Health Check** project successfully demonstrates the application of **AI-powered image analysis** to assist in early plant disease detection and health assessment. By integrating the **Streamlit framework** with **Google Gemini AI Studio**, the system efficiently analyzes plant leaf images to identify possible diseases, detect discoloration patterns, determine severity levels, and provide actionable rescue recommendations.

The application processes user-uploaded leaf images in real time, leveraging Gemini's advanced vision and reasoning capabilities to generate accurate and meaningful diagnosis reports. The intuitive and responsive Streamlit interface ensures ease of use, allowing users to quickly upload images, view analysis results, and download reports without technical complexity. The system emphasizes reliability, usability, and performance through structured testing, optimization, and effective error handling.

Overall, **Leaf Health Check** serves as a smart digital assistant for farmers, gardeners, and agricultural enthusiasts by enabling timely plant health assessment and informed decision-making. The project highlights the potential of AI-driven solutions in modern agriculture and lays a strong foundation for future enhancements such as multi-crop support, real-time camera integration, and advanced disease prediction models, contributing to sustainable and technology-driven farming practices.